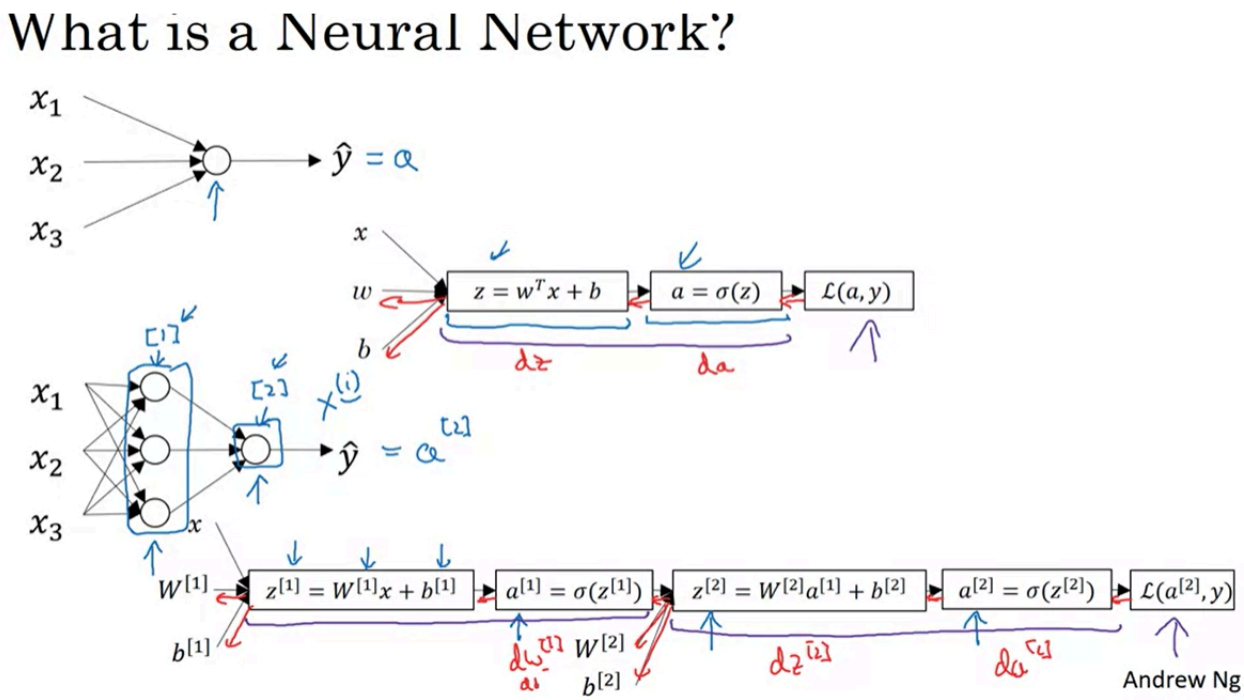
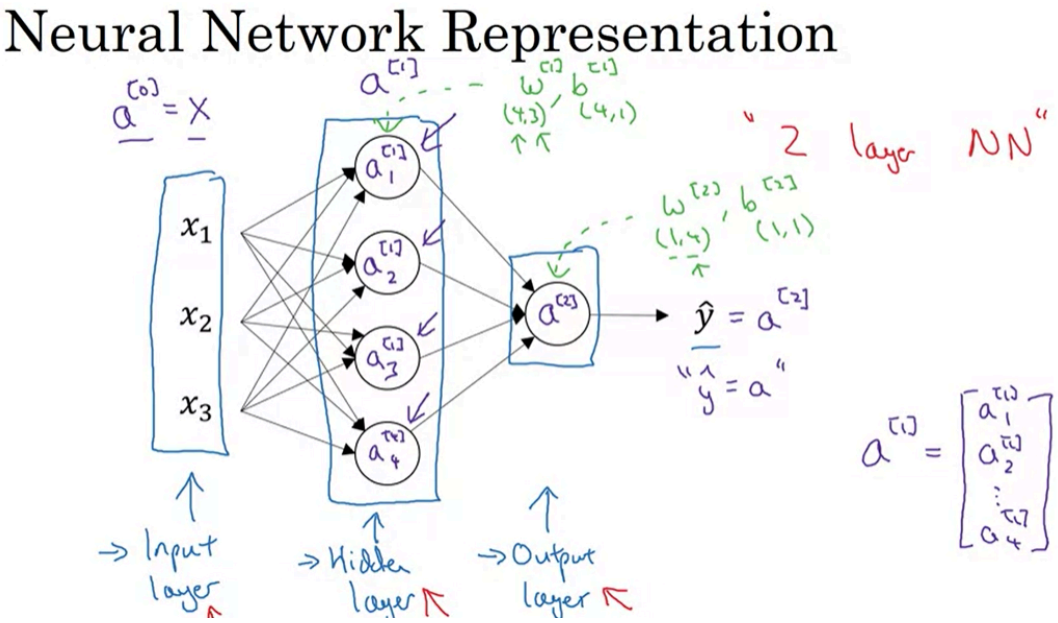
Shallow Neural Network

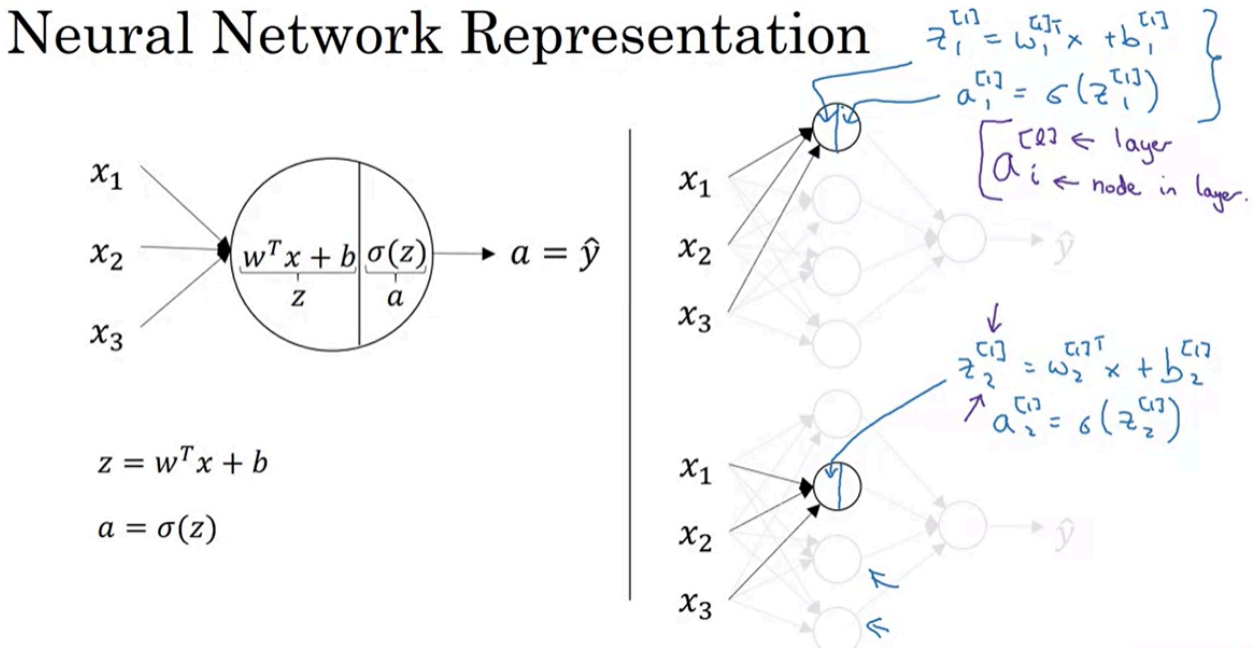
Neural Networks Overview

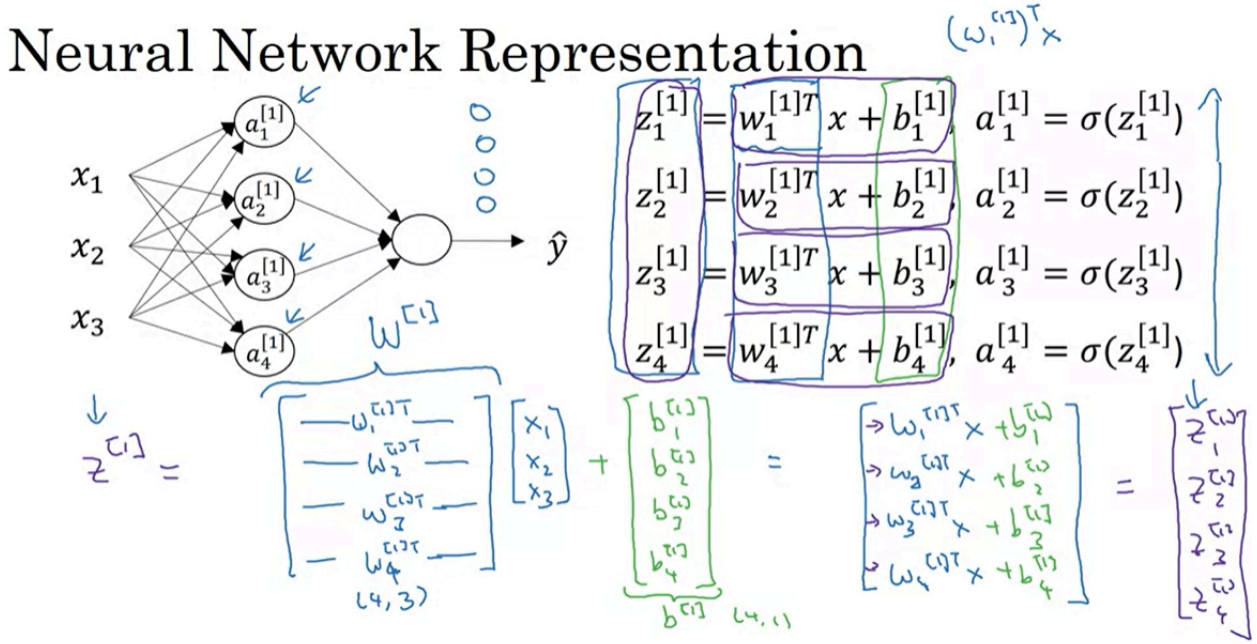


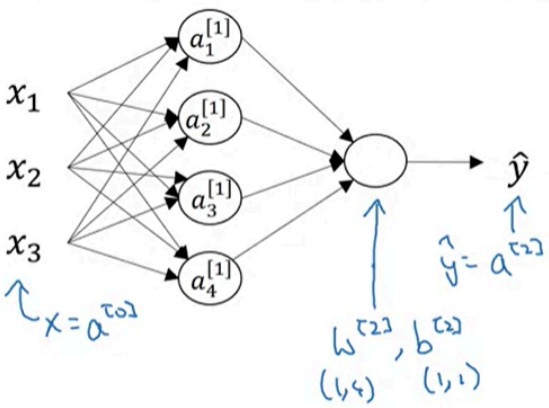
Neural Network Representation



Computing a Neural Network's Output







Given input x:

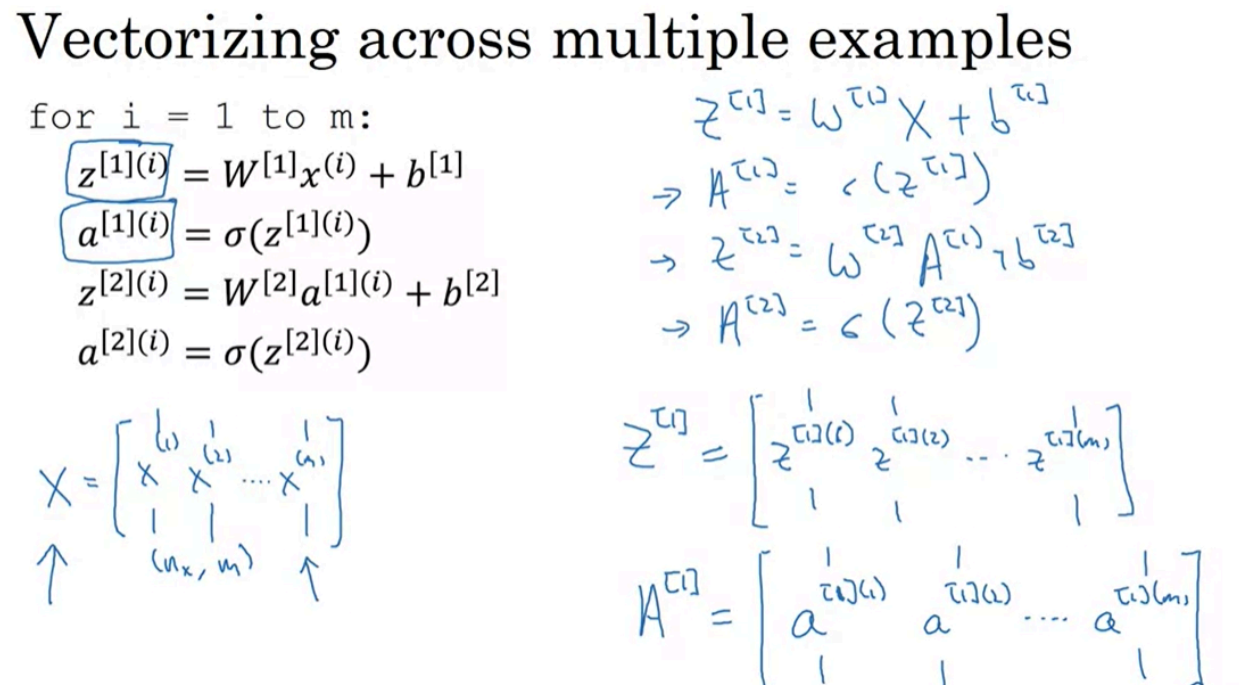
$$\rightarrow z^{[1]}_{(4,1)} = W^{[1]}_{(4,3)} a^{[0]}_{(3,1)} + b^{[1]}_{(4,1)}$$

$$\rightarrow a^{[1]}_{(4,1)} = \sigma(z^{[1]}_{(4,1)})$$

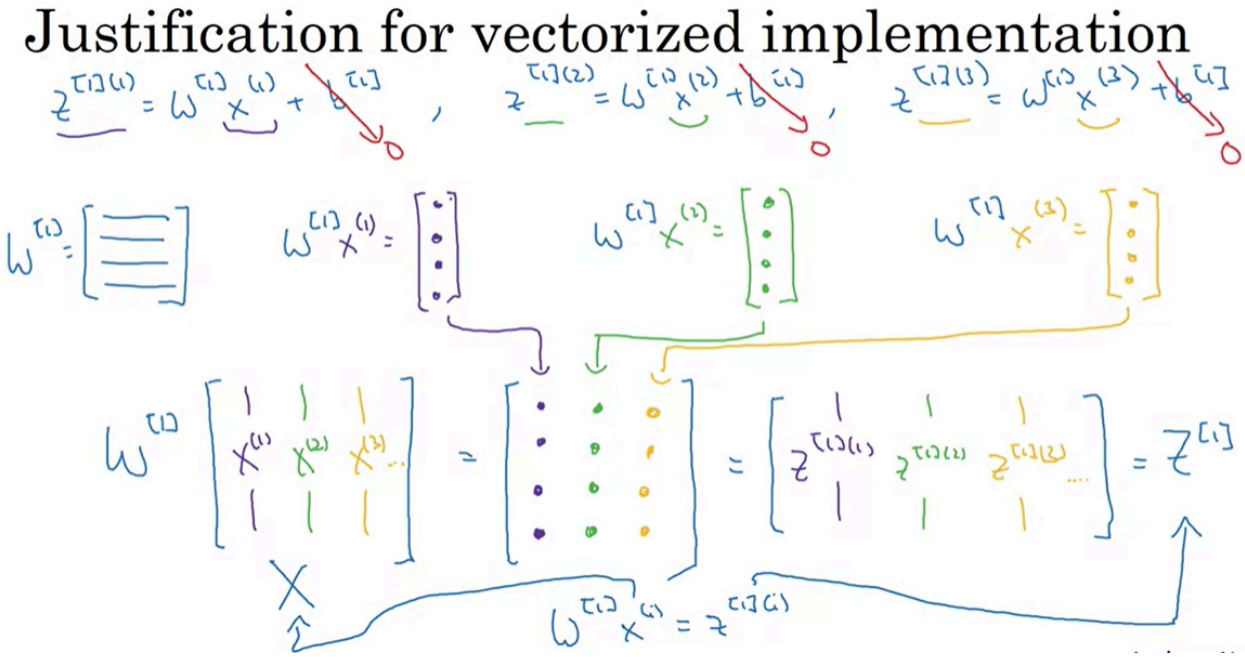
$$\rightarrow z^{[2]}_{(1,1)} = W^{[2]}_{(1,4)} a^{[1]}_{(4,1)} + b^{[2]}_{(1,1)}$$

$$\rightarrow a^{[2]}_{(1,1)} = \sigma(z^{[2]}_{(1,1)})$$

Vectorizing Across Multiple Examples

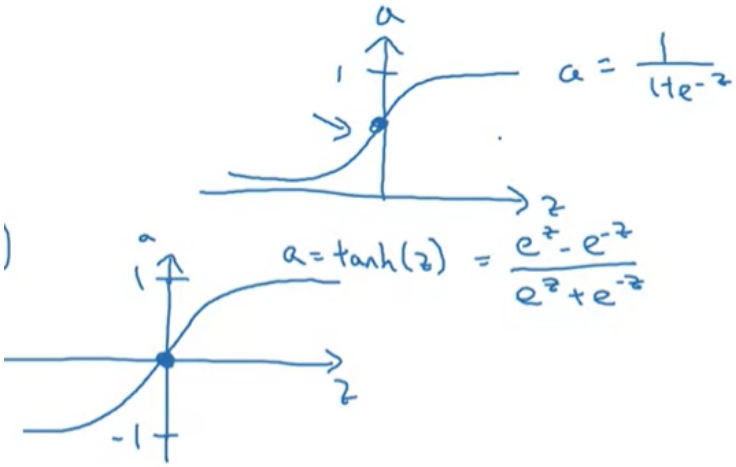


Explanation for Vectorized Implementation



Activation Functions

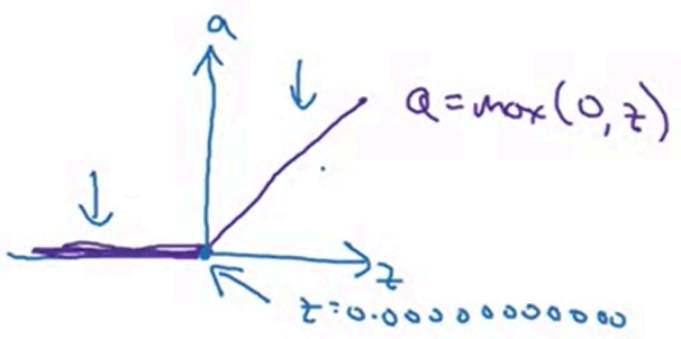
tanh(z) always work better than sigmoid activation function



But one takeaway is that I pretty much never use the sigmoid activation function anymore. The tan h function is almost always strictly superior. The one exception is for the output layer because if y is either zero or one, then it makes sense for y hat to be a number that you want to output that's between zero and one rather than between -1 and 1. So the one exception where I would use the sigmoid activation function is when you're using binary classification. Thats to only for the output layer.

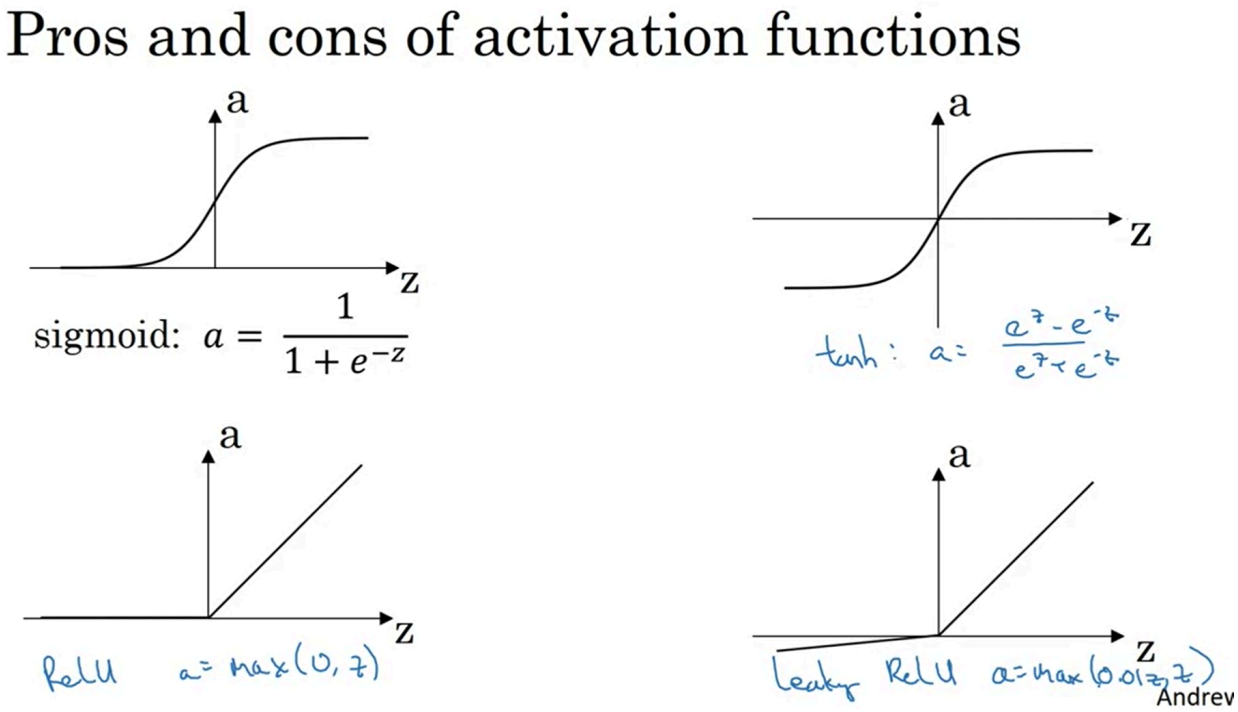
Now, one of the downsides of both the sigmoid function and the tan h function is that if z is either very large or very small, then the gradient of the derivative of the slope of this function becomes very small. So if z is very large or z is very small, the slope of the function either ends up being close to zero and so this can slow down gradient descent.

So one other choice that is very popular in machine learning is what's called the rectified linear unit. So the value function looks like this and the formula is a = max(0,z). So the derivative is one so long as z is positive and derivative or the slope is zero when z is negative. If you're implementing this, technically the derivative when z is exactly zero is not well defined. But when you implement this in the computer, the odds that you get exactly z equals 000000000000 is very small.



The fact that, so here's some rules of thumb for choosing activation functions. If your output is zero one value, if you're using binary classification, then the sigmoid activation function is very natural choice for the output layer. And then for all other units value or the rectified linear unit is increasingly the default choice of activation function.

One disadvantage of the value is that the derivative is equal to zero when z is negative. In practice this works just fine. But there is another version of the value called the Leaky ReLU.

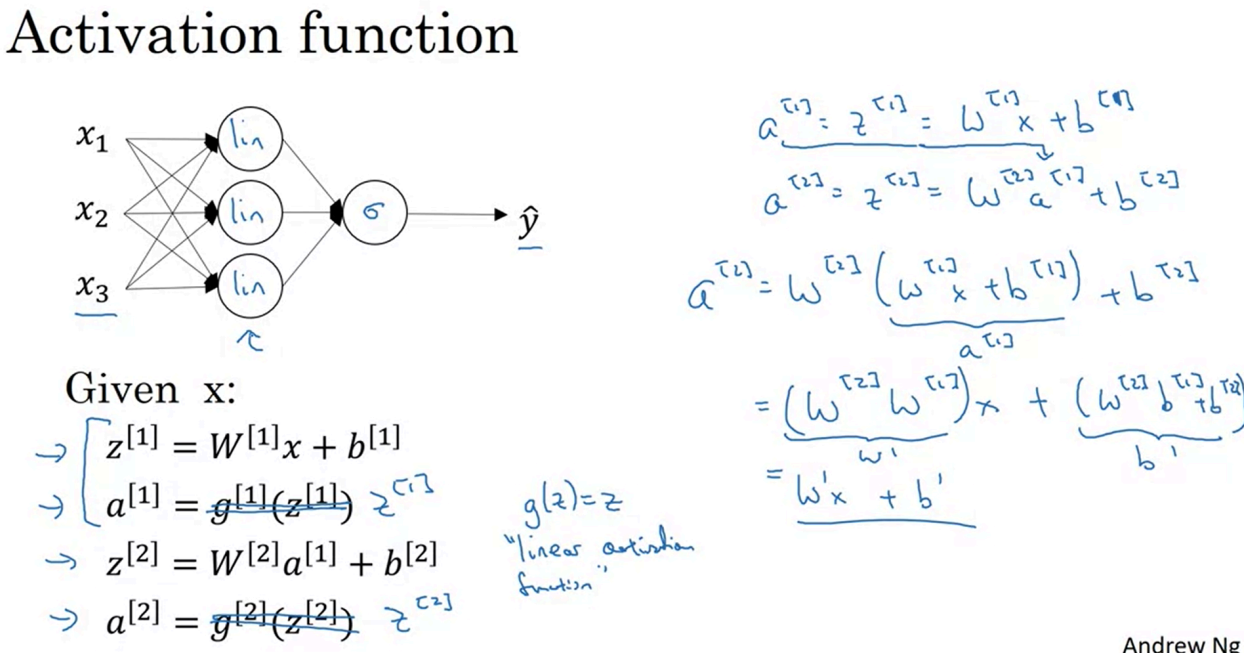


So in practice, using the ReLU activation function, your neural network will often learn much faster than when using the tan h or the sigmoid activation function. And the main reason is that there's less of this effect of the slope of the function going to zero, which slows down learning.

Why do you need Non-Linear Activation Functions?

If you were to use linear activation functions or we can also call them identity activation functions, then the neural network is just outputting a linear function of the input.

And it turns out that if you use a linear activation function or alternatively, if you don't have an activation function, then no matter how many layers your neural network has, all it's doing is just computing a linear activation function. So you might as well not have any hidden layers.

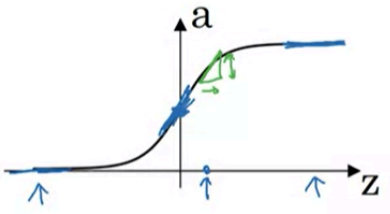


But the take home is that a linear hidden layer is more or less useless because the composition of two linear functions is itself a linear function. So unless you throw a non-linear item in there, then you're not computing more interesting functions even as you go deeper in the network.

The one place you might use a linear activation function is usually in the output layer. But other than that, using a linear activation function in the hidden layer except for some very special circumstances relating to compression that we're going to talk about using the linear activation function is extremely rare.

Derivatives of Activation Functions

Sigmoid activation function



$$g(z) = \frac{1}{1+e^{-z}}$$

$a = g(z) = \frac{1}{1+e^{-z}}$

$g'(z) = \frac{d}{dz} g(z)$ = slope of $g(x)$ at z

$= \frac{1}{1+e^{-z}} \left(1 - \frac{1}{1+e^{-z}}\right)$

$= g(z)\left(1 - g(z)\right) \leftarrow$

$= a\,(1-a)$ $\quad | \; g'(z) = a(1-a)$

$z = 10$. $g(z) \approx 1$

$\frac{d}{dz} g(z) \approx 1\,(1-1) \approx 0$

$z = -10$ $g(z) \approx 0$

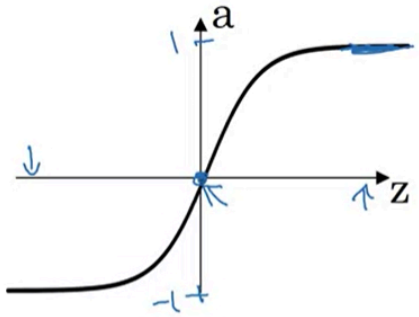
$\frac{d}{dz} g(z) \approx 0 \cdot (1-0) \approx 0$

$z = 0$ $g(z) = \frac{1}{2}$

$\frac{d}{dz} g(z) = \frac{1}{2}\left(1 - \frac{1}{2}\right) = \frac{1}{4}$

Andrew Ng

Tanh activation function



$$g(z) = \tanh(z)$$

$= \frac{e^{z} - e^{-z}}{e^{z} + e^{-z}}$

$g'(z) = \frac{d}{dz} g(z)$ = slope of $g(z)$ at z

$= 1 - (\tanh(z))^2 \leftarrow$

$a = g(z)$, $\quad g'(z) = 1 - a^2$

$z = 10$ $\quad \tanh(z) \approx 1$

$g'(z) \approx 0$

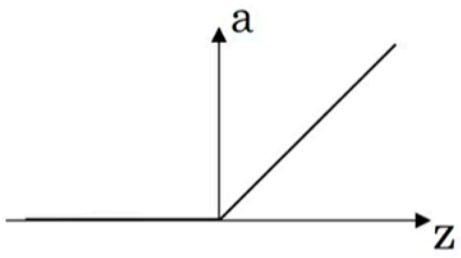
$z = -10$ $\quad \tanh(z) \approx -1$

$g'(z) \approx 0$

$z = 0$ $\quad \tanh(z) = 0$

$g'(z) = 1$

ReLU and Leaky ReLU

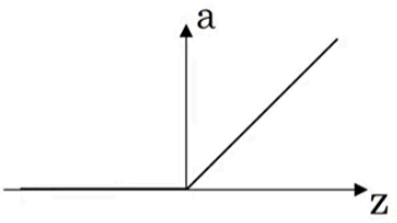


ReLU

$g(z) = \max(0, z)$

$$g'(z) = \begin{cases} 0 & \text{if } z < 0 \\ 1 & \text{if } z > 0 \\ \text{undefined} & \text{if } z = 0 \end{cases}$$

ReLU and Leaky ReLU

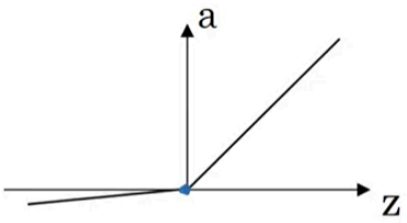


ReLU

$g(z) = \max(0, z)$

$$g'(z) = \begin{cases} 0 & \text{if } z < 0 \\ 1 & \text{if } z \geq 0 \\ \text{~~undefined if } z = 0 & \end{cases}$$

$z = 0.0000\ldots0$



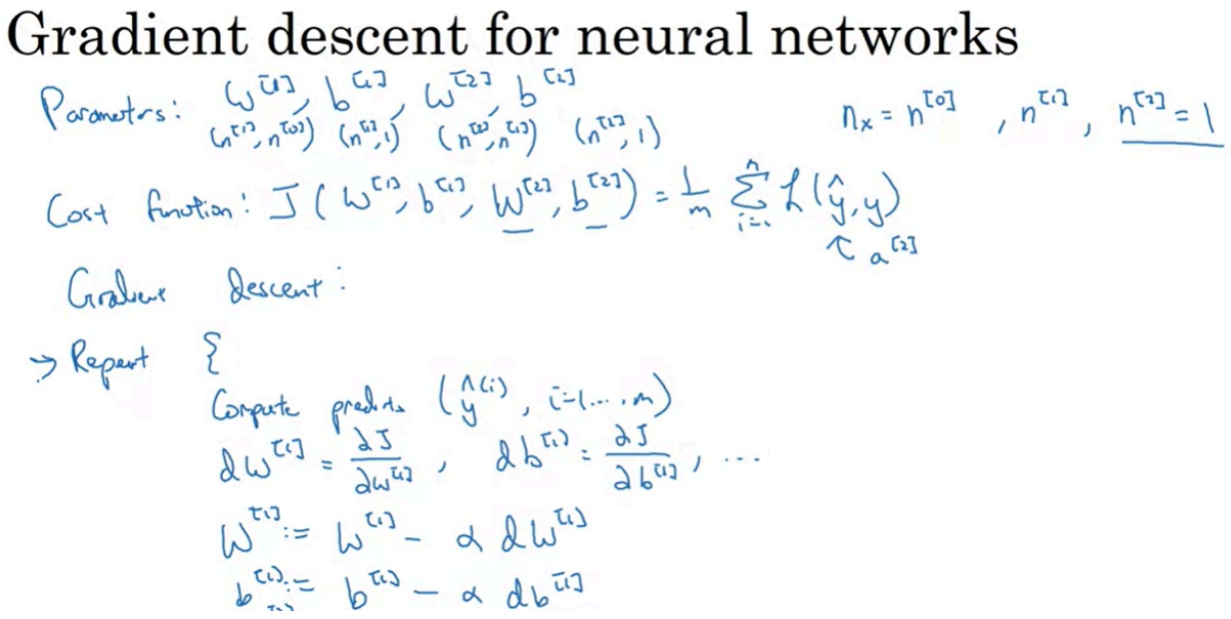
Leaky ReLU

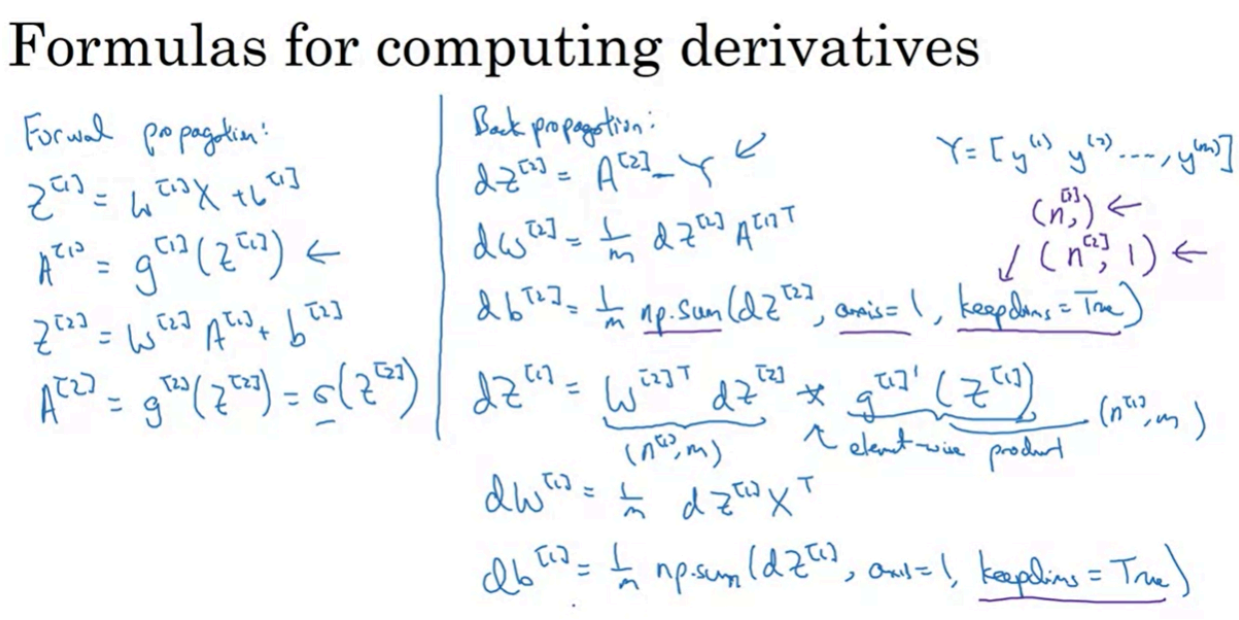
$g(z) = \max(0.01z, z)$

$$g'(z) = \begin{cases} 0.01 & \text{if } z < 0 \\ 1 & \text{if } z > 0 \end{cases}$$

Gradient Descent for Neural Networks

You'll see how to implement **gradient descent for your neural network with one hidden layer.** I'm going to just give you the **equations** you need to implement in order to get back-propagation or to get gradient descent working,





Backpropagation Intuition

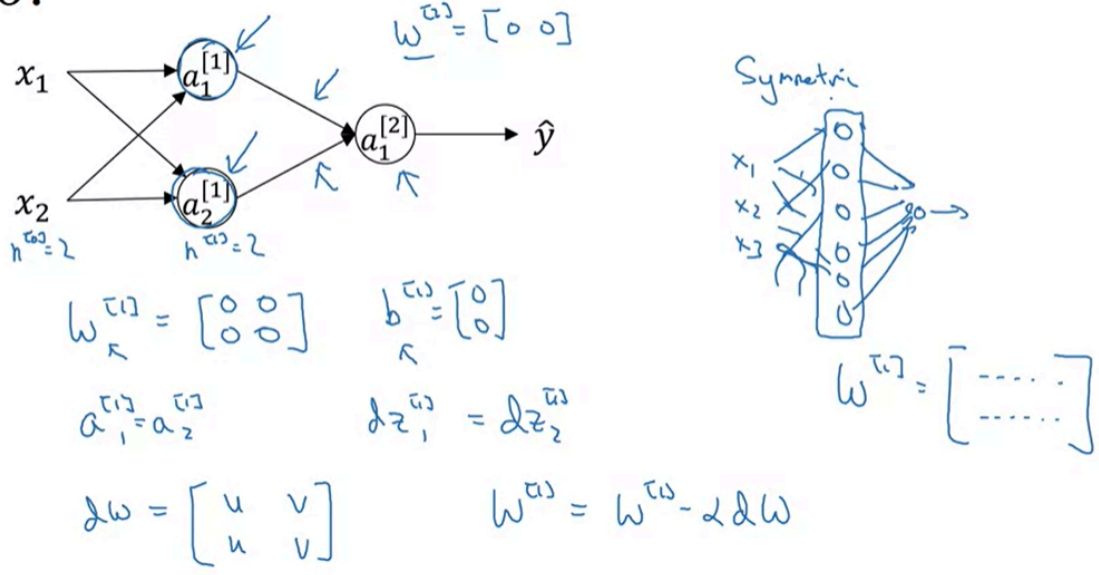
Summary of gradient descent

$dz^{[2]} = a^{[2]} - y$	$dZ^{[2]} = A^{[2]} - Y$
$dW^{[2]} = dz^{[2]} a^{[1]T}$	$dW^{[2]} = \frac{1}{m} dZ^{[2]} A^{[1]T}$
$db^{[2]} = dz^{[2]}$	$db^{[2]} = \frac{1}{m} np.sum(dZ^{[2]}, axis = 1, keepdims = True)$
$dz^{[1]} = W^{[2]T} dz^{[2]} * g^{[1]\prime}(z^{[1]})$	$dZ^{[1]} = W^{[2]T} dZ^{[2]} * g^{[1]\prime}(Z^{[1]})$
$dW^{[1]} = dz^{[1]} x^T$	$dW^{[1]} = \frac{1}{m} dZ^{[1]} X^T$
$db^{[1]} = dz^{[1]}$	$db^{[1]} = \frac{1}{m} np.sum(dZ^{[1]}, axis = 1, keepdims = True)$

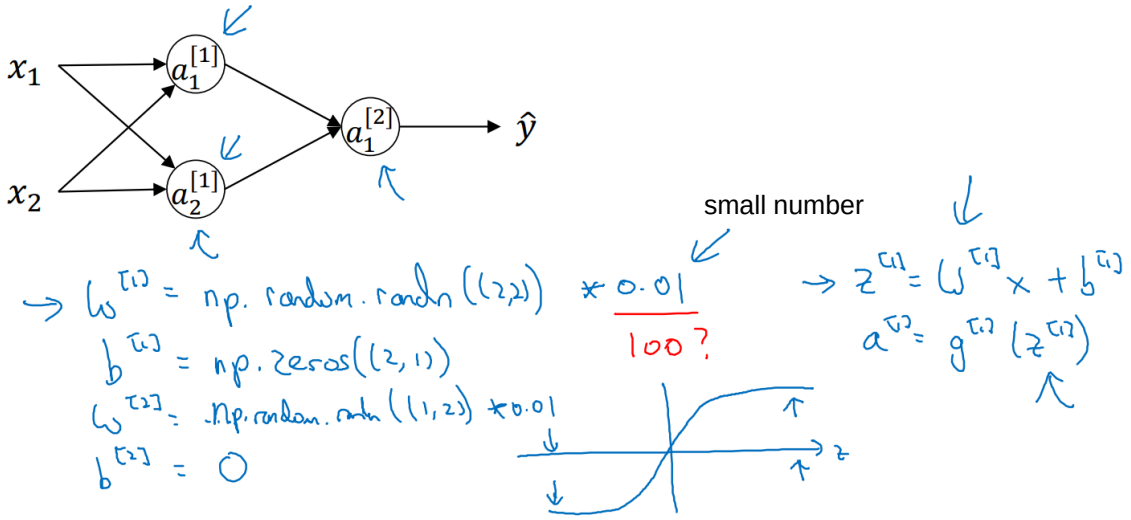
Random Initialization

When you change your neural network, it's important to initialize the weights randomly. For logistic regression, it was okay to initialize the weights to zero. But for a neural network of initialize the weights to parameters to all zero and then applied gradient descent, it won't work.

What happens if you initialize weights to zero?



Random initialization



If w is very big, z will be very, or at least some values of z will be either very large or very small. And so in that case, you're more likely to end up at these fat parts of the tanh function or the sigmoid function, where the slope or the gradient is very small.

It turns out initializing the bias terms b to 0 is actually okay, but initializing w to all 0s is a problem.

This is because, all the hidden units in a layer is the same, along with its derivative during backprop. Hence, the NN does the same task for all the iteation. They are computing the same function again and again, which doesn't make sense.

if you initialize the weights to zero, then all of your hidden units are symmetric. And no matter how long you're upgrading the gradients, all continue to compute exactly the same function.